In [1]:

```
%%writefile mantisv2_kit.py
# MantisV2 kit (same file as kit/mantisv2_kit.py in the group GitHub rep
o)
import argparse
import json
import os
import subprocess
import time
import traceback
import zipfile

import numpy as np
import pandas as pd
import requests
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

ALL_METHODS = ['rf', 'logreg', 'head', 'mixer', 'pool', 'pool_mixer', 'lo
ra']
SEEDS = (0, 1, 2)
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

TFC_ARTICLES = {'SleepEEG': 19930178, 'Epilepsy': 19930199, 'FD-A': 19930
205, 'FD-B': 19930226,
                'HAR': 19930244, 'Gesture': 19930247, 'EMG': 19930250, 'E
CG': 19930253}


def log(*msg):
    print(time.strftime('%H:%M:%S'), *msg, flush=True)


def _download(url, path):
    r = requests.get(url, timeout=900)
    r.raise_for_status()
```

```
    with open(path, 'wb') as fh:
        fh.write(r.content)


def _parse_ts(path):
    X, y, in_data = [], [], False
    with open(path, encoding='utf-8', errors='ignore') as fh:
        for line in fh:
            line = line.strip()
            if not line or line.startswith('#'):
                continue
            if line.lower().startswith('@data'):
                in_data = True
                continue
            if in_data:
                *channels, label = line.split(':')
                X.append([np.array([float(v) if v not in ('?', 'NaN') els
e np.nan for v in ch.split(',')],
                                   dtype=np.float32) for ch in channels])
                y.append(label)
    L = max(len(ch) for row in X for ch in row)
    out = np.zeros((len(X), len(X[0]), L), dtype=np.float32)
    for i, row in enumerate(X):
        for c, ch in enumerate(row):
            if len(ch) == L:
                out[i, c] = ch
            else:
                ch = np.nan_to_num(ch)
                out[i, c] = np.interp(np.linspace(0, len(ch) - 1, L), np.
arange(len(ch)), ch)
    return out, np.asarray(y)


def load_uea(name, data_dir):
    folder = os.path.join(data_dir, name)
    if not os.path.exists(folder):
        zip_path = os.path.join(data_dir, f'{name}.zip')
        log(f'downloading {name} from timeseriesclassification.com')
        _download(f'https://www.timeseriesclassification.com/aeon-toolki
t/{name}.zip', zip_path)
        with zipfile.ZipFile(zip_path) as z:
```

```
            z.extractall(folder)
        os.remove(zip_path)
    return load_ts_folder(name, folder)


def load_ts_folder(name, folder):
    out = {}
    for split in ['TRAIN', 'TEST']:
        found = [os.path.join(r, f) for r, _, fs in os.walk(folder) for f
in fs if f == f'{name}_{split}.ts']
        if not found:
            raise FileNotFoundError(f'{name}_{split}.ts not found in {fol
der} (the name must match exactly)')
        out[split.lower()] = _parse_ts(found[0])
    return out


def load_tfc(name, data_dir):
    folder = os.path.join(data_dir, f'TFC-{name}')
    os.makedirs(folder, exist_ok=True)
    files = None
    out = {}
    for split in ['train', 'test']:
        path = os.path.join(folder, f'{split}.pt')
        if not os.path.exists(path):
            if files is None:
                files = requests.get(f'https://api.figshare.com/v2/articl
es/{TFC_ARTICLES[name]}', timeout=30).json()['files']
            log(f'downloading TF-C {name} {split}.pt from figshare')
            _download([f['download_url'] for f in files if f['name'] ==
f'{split}.pt'][0], path)
        d = torch.load(path, weights_only=False)
        X, y = np.asarray(d['samples'], dtype=np.float32), np.asarray(d
['labels'])
        out[split] = (X, y)
    return out


def load_numpy(path):
    f = lambda n: os.path.join(path, n)
    if os.path.exists(f('X_train.npy')):
```

```
        return {'train': (np.load(f('X_train.npy')), np.load(f('y_train.n
py'), allow_pickle=True)),
                'test': (np.load(f('X_test.npy')), np.load(f('y_test.np
y'), allow_pickle=True))}
    X, y = np.load(f('X.npy')), np.load(f('y.npy'), allow_pickle=True)
    tr, te = train_test_split(np.arange(len(y)), test_size=0.25, stratify
=y, random_state=0)
    return {'train': (X[tr], y[tr]), 'test': (X[te], y[te])}


def load_pt(path):
    out = {}
    for split in ['train', 'test']:
        d = torch.load(os.path.join(path, f'{split}.pt'), weights_only=Fa
lse)
        out[split] = (np.asarray(d['samples'], dtype=np.float32), np.asar
ray(d['labels']))
    return out


def load_woods(path, split='ID', train_domains=None, test_domains=None, c
hannels_last=None):
    import h5py
    with h5py.File(path, 'r') as h:
        domains = sorted(h.keys())
        log(f'{os.path.basename(path)} domains: {domains}')
        if split == 'OOD':
            missing = [m for m in (train_domains or []) + (test_domains o
r []) if m not in domains]
            if missing or not train_domains or not test_domains:
                raise ValueError(f'for OOD give train_domains and test_do
mains from this list: {domains} '
                                 f'(not found: {missing})')
        Xs, ys, dom = [], [], []
        for k, m in enumerate(domains):
            if split == 'OOD' and m not in train_domains + test_domains:
                continue
            data, parts = h[f'{m}/data'], []
            last = channels_last if channels_last is not None else data.s
hape[1] > data.shape[2]
            for s in range(0, data.shape[0], 2000):
```

```
                x = data[s:s + 2000]
                parts.append(resize(np.transpose(x, (0, 2, 1)) if last el
se x))
            labels = h[f'{m}/labels'][:]
            labels = labels.argmax(1) if labels.ndim == 2 and labels.shap
e[1] > 1 else labels.ravel()
            Xs.append(np.concatenate(parts))
            ys.append(labels)
            dom += [m] * len(labels)
            log(f'  {m}: {Xs[-1].shape[0]} samples, {Xs[-1].shape[1]} cha
nnels')
    X, y, dom = np.concatenate(Xs), np.concatenate(ys), np.asarray(dom)
    if split == 'ID':
        N = len(y)
        perm = np.random.RandomState(0).permutation(N)
        tr, te = perm[:int(0.6375 * N)], perm[N - int(round(0.25 * N)):]
    else:
        tr, te = np.where(np.isin(dom, train_domains))[0], np.where(np.is
in(dom, test_domains))[0]
    return {'train': (X[tr], y[tr]), 'test': (X[te], y[te])}


SOURCES = ['uea', 'tfc', 'ts', 'numpy', 'pt', 'woods']


def load_dataset(name, source, path=None, data_dir='data', **kw):
    os.makedirs(data_dir, exist_ok=True)
    if source == 'uea':
        d = load_uea(name, data_dir)
    elif source == 'tfc':
        d = load_tfc(name, data_dir)
    elif source == 'ts':
        d = load_ts_folder(name, path or os.path.join(data_dir, name))
    elif source == 'numpy':
        d = load_numpy(path or os.path.join(data_dir, name))
    elif source == 'pt':
        d = load_pt(path or os.path.join(data_dir, name))
    elif source == 'woods':
        d = load_woods(path, **kw)
    else:
        raise ValueError(f'source must be one of {SOURCES}')
```

```
    for k, (X, y) in d.items():
        X = np.asarray(X)
        if X.dtype != np.float16:
            X = X.astype(np.float32)
        if X.ndim == 2:
            X = X[:, None, :]
        d[k] = (X, np.asarray(y).ravel())
    return d


_NET = None


def encoder():
    global _NET
    if _NET is None:
        from mantis.architecture import MantisV2
        net = MantisV2(device=DEVICE, return_transf_layer=2, output_token
='combined').from_pretrained('paris-noah/MantisV2')
        net.return_transf_layer, net.output_token = 2, 'combined'
        _NET = net.to(DEVICE).eval()
    return _NET


def resize(X, length=512):
    out = []
    for i in range(0, len(X), 2000):
        x = torch.tensor(np.nan_to_num(np.asarray(X[i:i + 2000], dtype=n
p.float32)))
        out.append(F.interpolate(x, size=length, mode='linear', align_cor
ners=False).numpy().astype(np.float16))
    return np.concatenate(out)


def features(Xr):
    from mantis.trainer import MantisTrainer
    model = MantisTrainer(device=DEVICE, network=encoder())
    N, C, _ = Xr.shape
    out = [model.transform(Xr[i:i + 2000].astype(np.float32)) for i in ra
nge(0, N, 2000)]
    return np.concatenate(out).reshape(N, C, 512).astype(np.float16)
```

```
@torch.no_grad()
def tokens(Xr, bs=1024):
    net = encoder()
    N, C, _ = Xr.shape
    flat = Xr.reshape(N * C, 1, 512)
    out = []
    for i in range(0, N * C, bs):
        h = net.tokgen_unit(torch.from_numpy(flat[i:i + bs]).to(DEVICE).f
loat())
        h = torch.cat([h, net.transf_unit.cls_token.expand(h.shape[0], 1,
-1)], 1)[:, None]
        layers = []
        for layer in net.transf_unit.transformer.layers:
            h = layer(h)
            t = h[:, 0]
            layers.append(torch.cat([t[:, 32:], t[:, :32].reshape(-1, 4,
8, 256).mean(2)], 1))
        out.append(torch.stack(layers, 1).half().cpu().numpy())
    return np.concatenate(out).reshape(N, C, 6, 5, 256)


class MixerV2(nn.Module):
    def __init__(self, C, K, mix, d=64, drop=0.3, chan_drop=0.1, d_in=51
2):
        super().__init__()
        self.mix, self.chan_drop = mix, chan_drop
        self.proj = nn.Sequential(nn.LayerNorm(d_in), nn.Linear(d_in, d))
        if mix:
            self.chan_emb = nn.Parameter(torch.zeros(1, C, d))
            self.norm = nn.LayerNorm(d)
            self.attn = nn.MultiheadAttention(d, 2, dropout=drop, batch_f
irst=True)
            self.gate = nn.Parameter(torch.zeros(1))
        self.out = nn.Sequential(nn.Dropout(drop), nn.Linear(C * d, K))

    def forward(self, Z):
        z = self.proj(Z.float())
        if self.training and self.chan_drop > 0:
            z = z * (torch.rand(z.shape[0], z.shape[1], 1, device=z.devic
```

```
e) > self.chan_drop).float()
        if self.mix:
            h = self.norm(z + self.chan_emb)
            z = z + torch.tanh(self.gate) * self.attn(h, h, h, need_weigh
ts=False)[0]
        return self.out(z.flatten(1))


class PoolHead(nn.Module):
    def __init__(self, C, K, mix, L=6, heads=4):
        super().__init__()
        self.layer_w = nn.Parameter(torch.zeros(L))
        self.norm = nn.LayerNorm(256)
        self.query = nn.Parameter(torch.randn(heads, 256) * 0.02)
        self.head = MixerV2(C, K, mix, d_in=heads * 256)

    def forward(self, T):
        w = torch.softmax(self.layer_w, 0)
        t = (self.norm(T.float()) * w[:, None, None]).sum(2)
        a = torch.softmax(t @ self.query.T / 16, 2)
        return self.head((a.transpose(2, 3) @ t).flatten(2))


class LoRA(nn.Module):
    def __init__(self, base, r=8, alpha=16):
        super().__init__()
        self.base, self.scale = base, alpha / r
        self.A = nn.Parameter(torch.randn(r, base.in_features, device=bas
e.weight.device) * 0.01)
        self.B = nn.Parameter(torch.zeros(base.out_features, r, device=ba
se.weight.device))

    def forward(self, x):
        return self.base(x) + (x @ self.A.T @ self.B.T) * self.scale


class LoraModel(nn.Module):
    def __init__(self, C, K, r=8):
        super().__init__()
        from mantis.architecture import MantisV2
        self.enc = MantisV2(device=DEVICE, return_transf_layer=2, output_
```

```
token='combined')
        self.enc.load_state_dict(encoder().state_dict())
        self.enc.remove_transf_layers()
        for p in self.enc.parameters():
            p.requires_grad = False
        for layer in self.enc.transf_unit.transformer.layers:
            layer.attention.wQKV = LoRA(layer.attention.wQKV, r)
            layer.attention.wO = LoRA(layer.attention.wO, r)
        self.head = MixerV2(C, K, True)

    def forward(self, X):
        B, C, L = X.shape
        return self.head(self.enc(X.reshape(B * C, 1, L).float()).reshape
(B, C, 512))


def fit(model, A, y, tr_idx, te_idx, seed, epochs, patience, bs, lr=1e-3,
amp=False):
    amp = amp and DEVICE == 'cuda'
    fit_idx, va_idx = train_test_split(tr_idx, test_size=0.2, stratify=y
[tr_idx], random_state=seed)
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.AdamW(params, lr=lr, weight_decay=0.05)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, epochs)
    scaler = torch.amp.GradScaler('cuda', enabled=amp)
    batch = lambda idx: torch.from_numpy(A[idx]).to(DEVICE)

    def predict(idx):
        model.eval()
        preds = []
        with torch.no_grad(), torch.autocast(DEVICE, dtype=torch.float16,
enabled=amp):
            for i in range(0, len(idx), 256):
                preds.append(model(batch(idx[i:i + 256])).argmax(1).cpu
().numpy())
        return np.concatenate(preds)

    best, best_state, wait = -1, None, 0
    for ep in range(epochs):
        model.train()
        perm = np.random.permutation(fit_idx)
```

```
        for i in range(0, len(perm), bs):
            b = perm[i:i + bs]
            with torch.autocast(DEVICE, dtype=torch.float16, enabled=am
p):
                loss = F.cross_entropy(model(batch(b)), torch.from_numpy
(y[b]).long().to(DEVICE), label_smoothing=0.1)
            opt.zero_grad()
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
        sched.step()
        va = np.mean(predict(va_idx) == y[va_idx])
        if va >= best:
            best, wait = va, 0
            best_state = {k: v.detach().clone() for k, v in model.state_d
ict().items()}
        else:
            wait += 1
            if wait >= patience:
                break
    model.load_state_dict(best_state)
    return float(np.mean(predict(te_idx) == y[te_idx])), ep + 1


def settings(n_train, quick=False):
    if quick:
        return dict(head=dict(epochs=3, patience=2, bs=64), pool=dict(epo
chs=2, patience=1, bs=64),
                    lora=dict(epochs=1, patience=1, bs=32))
    if n_train < 10000:
        return dict(head=dict(epochs=100, patience=20, bs=64), pool=dict
(epochs=100, patience=20, bs=64),
                    lora=dict(epochs=30, patience=8, bs=32))
    return dict(head=dict(epochs=40, patience=8, bs=256), pool=dict(epoch
s=20, patience=5, bs=256),
                lora=dict(epochs=4, patience=2, bs=64))


def run(dataset, source='uea', path=None, methods=None, seeds=SEEDS, out_
dir='results', data_dir='data',
        quick=False, paper=None, **load_kw):
```

```
    methods = methods or ALL_METHODS
    os.makedirs(out_dir, exist_ok=True)
    csv = os.path.join(out_dir, f'{dataset}_results.csv')
    t_all = time.time()
    log(f'{dataset}: device {DEVICE}' + (f' ({torch.cuda.get_device_name
(0)})' if DEVICE == 'cuda' else
                                          ' - no GPU, the trained methods
will be slow'))

    d = load_dataset(dataset, source, path, data_dir, **load_kw)
    (Xa, ya), (Xb, yb) = d['train'], d['test']
    le = LabelEncoder().fit(np.concatenate([ya, yb]))
    y = le.transform(np.concatenate([ya, yb]))
    tr, te = np.arange(len(ya)), np.arange(len(ya), len(ya) + len(yb))
    L0 = Xa.shape[2]
    Xr = resize(np.concatenate([Xa, Xb]))
    del Xa, Xb, d
    N, C, _ = Xr.shape
    K = len(le.classes_)
    S = settings(len(tr), quick)
    log(f'{dataset}: train {len(tr)}, test {len(te)}, channels {C}, origi
nal length {L0}, classes {K}')

    rows = []

    def record(method, accs, t0, **extra):
        row = dict(dataset=dataset, method=method, mean=float(np.mean(acc
s)), std=float(np.std(accs)),
                   per_seed=json.dumps([round(a, 4) for a in accs]), pape
r=paper, n_train=len(tr), n_test=len(te),
                   channels=C, classes=K, time_s=round(time.time() - t0),
**extra)
        rows.append(row)
        pd.DataFrame(rows).to_csv(csv, index=False)
        log(f'{dataset:16s} {method:12s} {row["mean"]:.4f} +/- {row["st
d"]:.4f}  {extra or ""}  {row["time_s"]}s')

    def safe(fn):
        try:
            fn()
        except Exception as e:
```

```
            traceback.print_exc()
            log('FAILED:', type(e).__name__, e)

    Z = None
    if {'rf', 'logreg', 'head', 'mixer'} & set(methods):
        t0 = time.time()
        Z = features(Xr)
        log(f'features {Z.shape} in {time.time() - t0:.0f}s')
        Zflat = Z.reshape(N, -1).astype(np.float32)

    if 'rf' in methods:
        def m():
            t0, accs = time.time(), []
            for s in seeds:
                rf = RandomForestClassifier(n_estimators=200, n_jobs=-1,
random_state=s).fit(Zflat[tr], y[tr])
                accs.append(float(np.mean(rf.predict(Zflat[te]) == y[t
e])))
            record('rf', accs, t0)
        safe(m)

    if 'logreg' in methods:
        def m():
            t0 = time.time()
            sc = StandardScaler().fit(Zflat[tr])
            clf = LogisticRegression(max_iter=500).fit(sc.transform(Zflat
[tr]), y[tr])
            record('logreg', [float(np.mean(clf.predict(sc.transform(Zfla
t[te])) == y[te]))], t0)
        safe(m)

    def heads(name, A, make, cfg, extra_fn, amp=False):
        def m():
            t0, accs, eps, extras = time.time(), [], [], []
            for s in seeds:
                torch.manual_seed(s)
                np.random.seed(s)
                model = make().to(DEVICE)
                acc, ep = fit(model, A, y, tr, te, s, amp=amp, **cfg)
                accs.append(acc)
                eps.append(ep)
```

```
                extras.append(extra_fn(model))
            record(name, accs, t0, epochs=json.dumps(eps), **{k: json.dum
ps([e[k] for e in extras]) for k in extras[0]})
        safe(m)

    gate = lambda h: round(torch.tanh(h.gate).item(), 3) if h.mix else 0.
0
    if 'head' in methods:
        heads('head', Z, lambda: MixerV2(C, K, False), S['head'], lambda
m: {})
    if 'mixer' in methods:
        heads('mixer', Z, lambda: MixerV2(C, K, True), S['head'], lambda
m: {'gate': gate(m)})
    del Z

    if {'pool', 'pool_mixer'} & set(methods):
        gb = N * C * 6 * 5 * 256 * 2 / 1e9
        if gb > 20:
            log(f'skipping pool methods: tokens would need {gb:.0f} GB of
RAM')
        else:
            t0 = time.time()
            T = tokens(Xr)
            log(f'tokens {T.shape} ({gb:.1f} GB) in {time.time() - t0:.0
f}s')
            lw = lambda m: {'gate': gate(m.head),
                            'layer_weights': [round(v, 3) for v in torch.
softmax(m.layer_w, 0).tolist()]}
            if 'pool' in methods:
                heads('pool', T, lambda: PoolHead(C, K, False), S['poo
l'], lw)
            if 'pool_mixer' in methods:
                heads('pool_mixer', T, lambda: PoolHead(C, K, True), S['p
ool'], lw)
            del T

    if 'lora' in methods:
        heads('lora', Xr, lambda: LoraModel(C, K), S['lora'], lambda m:
{'gate': gate(m.head)}, amp=True)

    df = pd.DataFrame(rows)
```

```
    log(f'{dataset} done in {(time.time() - t_all) / 60:.1f} min, saved {csv}')
    print(df[['dataset', 'method', 'mean', 'std', 'per_seed']].round(4).to_string(index=False))
    return df
```

```
Writing mantisv2_kit.py
```

In [2]:

```
# Kaggle runner for datasets 1-6 and 13-18 (cell 2 of the notebook; cell
1 is "%%writefile mantisv2_kit.py" + the kit).
# The same notebook is committed several times with a different PART so t
he parts run in parallel on Kaggle:
#   PART 1: SelfRegulationSCP1, SelfRegulationSCP2, EMOPain, MP8, MP50, H
HAR-ID, HHAR-OOD
#   PART 2: PCL-ID, PCL-OOD        PART 3: SEDFx-ID        PART 4: SEDFx-
OOD
#   PART 5: re-runs after two memory fixes (LoRA batch in fit(), token ar
ray in tokens()): LoRA for EMOPain and MP50,
#           pool/pool_mixer/LoRA for PCL-ID, everything for PCL-OOD (part
2 ran out of RAM), idea 2 for Blink/FingerMovements
# Ego4D (dataset 1) is not run: it needs a signed licence agreement.
PART = 4
!pip install -q mantis-tsfm gdown
import os, zipfile, subprocess, glob, numpy as np, pandas as pd, gdown
import mantisv2_kit as kit
# Memory fixes (same code as kit/mantisv2_kit.py on GitHub): LoRA batches
capped by the number of channels,
# and the token array filled in place
from mantisv2_kit import *
def fit(model, A, y, tr_idx, te_idx, seed, epochs, patience, bs, lr=1e-3,
amp=False):
    amp = amp and DEVICE == 'cuda'
    pb = 256  # prediction batch (samples)
    if isinstance(model, LoraModel):
        # LoRA runs the encoder on batch x channels series, so cap both b
atches by the number of channels
        C = A.shape[1]
        bs = max(4, min(bs, 1280 // C))
        pb = max(8, min(256, 4096 // C))
    fit_idx, va_idx = train_test_split(tr_idx, test_size=0.2, stratify=y
[tr_idx], random_state=seed)
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.AdamW(params, lr=lr, weight_decay=0.05)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, epochs)
    scaler = torch.amp.GradScaler('cuda', enabled=amp)
    batch = lambda idx: torch.from_numpy(A[idx]).to(DEVICE)

    def predict(idx):
```

```
        model.eval()
        preds = []
        with torch.no_grad(), torch.autocast(DEVICE, dtype=torch.float16,
enabled=amp):
            for i in range(0, len(idx), pb):
                preds.append(model(batch(idx[i:i + pb])).argmax(1).cpu().
numpy())
        return np.concatenate(preds)

    best, best_state, wait = -1, None, 0
    for ep in range(epochs):
        model.train()
        perm = np.random.permutation(fit_idx)
        for i in range(0, len(perm), bs):
            b = perm[i:i + bs]
            with torch.autocast(DEVICE, dtype=torch.float16, enabled=am
p):
                loss = F.cross_entropy(model(batch(b)), torch.from_numpy
(y[b]).long().to(DEVICE), label_smoothing=0.1)
            opt.zero_grad()
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
        sched.step()
        va = np.mean(predict(va_idx) == y[va_idx])
        if va >= best:
            best, wait = va, 0
            best_state = {k: v.detach().clone() for k, v in model.state_d
ict().items()}
        else:
            wait += 1
            if wait >= patience:
                break
    model.load_state_dict(best_state)
    return float(np.mean(predict(te_idx) == y[te_idx])), ep + 1
@torch.no_grad()
def tokens(Xr, bs=1024):
    net = encoder()
    N, C, _ = Xr.shape
    flat = Xr.reshape(N * C, 1, 512)
    out = np.empty((N * C, 6, 5, 256), dtype=np.float16)  # filled in pla
```

```
ce: a list + concatenate needs 2x the RAM
    for i in range(0, N * C, bs):
        h = net.tokgen_unit(torch.from_numpy(flat[i:i + bs]).to(DEVICE).f
loat())
        h = torch.cat([h, net.transf_unit.cls_token.expand(h.shape[0], 1,
-1)], 1)[:, None]
        layers = []
        for layer in net.transf_unit.transformer.layers:
            h = layer(h)
            t = h[:, 0]
            layers.append(torch.cat([t[:, 32:], t[:, :32].reshape(-1, 4,
8, 256).mean(2)], 1))
        out[i:i + bs] = torch.stack(layers, 1).half().cpu().numpy()
    return out.reshape(N, C, 6, 5, 256)
kit.fit, kit.tokens = fit, tokens
DATA, OUT = '/tmp/data', '/kaggle/working/results'
os.makedirs(DATA, exist_ok=True)

def get(fn):
    try:
        fn()
    except Exception as e:
        print('DATA FAILED:', type(e).__name__, e, flush=True)

def uea(n):
    subprocess.run(['wget', '-q', '-O', f'{DATA}/{n}.zip', f'https://www.
timeseriesclassification.com/aeon-toolkit/{n}.zip'], check=True)
    zipfile.ZipFile(f'{DATA}/{n}.zip').extractall(f'{DATA}/{n}')

def mp():
    # MP8 / MP50 as released by Sungu Isiacik et al. (2025), github.com/m
lgig/FM4MTSC
    gdown.download_folder(id='10Hvep0hk9naj_nRk4JMiXHMeV1vw0lcd', output=
f'{DATA}/fm4mtsc', quiet=True)
    for n in ['MP8', 'MP50']:
        d = np.load(f'{DATA}/fm4mtsc/{n}.npy', allow_pickle=True).item()
        print(n, list(d), {k: np.shape(v['X']) for k, v in d.items()}, fl
ush=True)
        os.makedirs(f'{DATA}/{n}', exist_ok=True)
        for s in ['train', 'test']:
            np.save(f'{DATA}/{n}/X_{s}.npy', np.asarray(d[s]['X'], dtype=
```

```
np.float32))
            np.save(f'{DATA}/{n}/y_{s}.npy', np.asarray(d[s]['y']))

def woods(gid, name):
    # WOODS benchmark files (github.com/jc-audet/WOODS, woods/download.p
y)
    gdown.download(id=gid, output=f'{DATA}/{name}.zip', quiet=True)
    zipfile.ZipFile(f'{DATA}/{name}.zip').extractall(f'{DATA}/{name}')
    os.remove(f'{DATA}/{name}.zip')

def h5(name):
    return [os.path.join(r, f) for r, _, fs in os.walk(f'{DATA}/{name}')
for f in fs if f.endswith('.h5')][0]

if PART == 1:
    for n in ['SelfRegulationSCP1', 'SelfRegulationSCP2', 'EMOPain']:
        get(lambda: uea(n))
    get(mp)
    get(lambda: woods('1Z3IcrCE-o77p4YrvkCy-Y-0CgCyxVHet', 'HHAR'))
    JOBS = [dict(name='SelfRegulationSCP1', source='uea', paper=0.8134),
            dict(name='SelfRegulationSCP2', source='uea', paper=0.5167),
            dict(name='EMOPain', source='uea', paper=0.8798),
            dict(name='MP8', source='numpy', paper=0.6857),
            dict(name='MP50', source='numpy', paper=0.7345),
            dict(name='HHAR-ID', source='woods', h5='HHAR', split='ID', p
aper=0.9845),
            dict(name='HHAR-OOD', source='woods', h5='HHAR', split='OOD',
train_domains=['nexus4', 's3', 's3mini'],
                 test_domains=['lgwatch'], paper=0.5822)]
elif PART == 5:
    # re-runs after the memory fixes (part 1: LoRA out of GPU memory; par
t 2: tokens out of RAM)
    for n in ['EMOPain', 'Blink', 'FingerMovements']:
        get(lambda: uea(n))
    get(mp)
    get(lambda: woods('118DNxHpzeJwVTM22wzZhSiOuDsno0nay', 'PCL'))
    JOBS = [dict(name='EMOPain', source='uea', methods=['lora'], paper=0.
8798),
            dict(name='MP50', source='numpy', methods=['lora'], paper=0.7
345),
            dict(name='PCL-ID', source='woods', h5='PCL', split='ID', met
```

```
hods=['pool', 'pool_mixer', 'lora'], paper=0.5716),
            dict(name='PCL-OOD', source='woods', h5='PCL', split='OOD', t
rain_domains=['Cho2017'],
                 test_domains=['Lee2019_MI'], paper=0.5311),
            # idea 2 with the current head for two of our datasets 7-12
(the site refuses downloads from our laptop)
            dict(name='Blink', source='uea', methods=['pool', 'pool_mixe
r'], paper=0.9956),
            dict(name='FingerMovements', source='uea', methods=['pool',
'pool_mixer'], paper=0.5500)]
elif PART == 2:
    get(lambda: woods('118DNxHpzeJwVTM22wzZhSiOuDsno0nay', 'PCL'))
    JOBS = [dict(name='PCL-ID', source='woods', h5='PCL', split='ID', pap
er=0.5716),
            dict(name='PCL-OOD', source='woods', h5='PCL', split='OOD', t
rain_domains=['Cho2017'],
                 test_domains=['Lee2019_MI'], paper=0.5311)]
else:
    get(lambda: woods('15j_bsiOmMJb42mG712Vhv3jZ4MQSOgoT', 'SEDFx'))
    JOBS = [dict(name='SEDFx-ID', source='woods', h5='SEDFx', split='ID',
paper=0.8000)] if PART == 3 else \
           [dict(name='SEDFx-OOD', source='woods', h5='SEDFx', split='OO
D', train_domains=['Age 20-40', 'Age 40-60'],
                 test_domains=['Age 80-100'], paper=0.7636)]

print(sorted(os.listdir(DATA)), flush=True)
for j in JOBS:
    j = dict(j)
    try:
        if 'h5' in j:
            j['path'] = h5(j.pop('h5'))
        kit.run(j.pop('name'), j.pop('source'), j.pop('path', None), out_
dir=OUT, data_dir=DATA, **j)
    except Exception as e:
        print('RUN FAILED:', type(e).__name__, e, flush=True)
files = glob.glob(f'{OUT}/*_results.csv')
if files:
    df = pd.concat([pd.read_csv(f) for f in files])
    df.to_csv(f'{OUT}/part{PART}_all_results.csv', index=False)
    print(df.pivot_table(index='dataset', columns='method', values='mea
n').round(4).to_string())
```

```
['SEDFx']
02:10:34 SEDFx-OOD: device cuda (Tesla T4)
02:10:34 SEDFx.h5 domains: ['Age 20-40', 'Age 40-60', 'Age 60-80', 'Ag
e 80-100']
02:10:53   Age 20-40: 64264 samples, 4 channels
02:11:07   Age 40-60: 69482 samples, 4 channels
02:11:17   Age 80-100: 52838 samples, 4 channels
02:11:22 SEDFx-OOD: train 133746, test 52838, channels 4, original len
gth 512, classes 6

Warning: You are sending unauthenticated requests to the HF Hub. Pleas
e set a HF_TOKEN to enable higher rate limits and faster downloads.




02:13:25 features (186584, 4, 512) in 123s
03:10:02 SEDFx-OOD        rf           0.7640 +/- 0.0002    3395s

/usr/local/lib/python3.13/dist-packages/sklearn/linear_model/_logisti
c.py:465: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as show
n in:
    https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
    https://scikit-learn.org/stable/modules/linear_model.html#logistic
-regression
  n_iter_i = _check_optimize_result(
```

```
03:15:43 SEDFx-OOD        logreg       0.7840 +/- 0.0000    341s
03:17:48 SEDFx-OOD        head         0.7928 +/- 0.0004  {'epochs':
'[40, 29, 40]'}  125s
03:21:18 SEDFx-OOD        mixer        0.7979 +/- 0.0006  {'epochs':
'[39, 40, 40]', 'gate': '[-0.155, -0.16, -0.147]'}  210s
03:24:25 tokens (186584, 4, 6, 5, 256) (11.5 GB) in 187s
03:28:36 SEDFx-OOD        pool         0.7909 +/- 0.0006  {'epochs':
'[20, 20, 20]', 'gate': '[0.0, 0.0, 0.0]', 'layer_weights': '[[0.207,
0.233, 0.239, 0.121, 0.102, 0.098], [0.207, 0.228, 0.239, 0.123, 0.10
4, 0.099], [0.208, 0.229, 0.242, 0.12, 0.102, 0.099]]'}  251s
03:33:18 SEDFx-OOD        pool_mixer   0.7962 +/- 0.0002  {'epochs':
'[20, 20, 20]', 'gate': '[-0.245, 0.282, 0.282]', 'layer_weights':
'[[0.216, 0.239, 0.229, 0.118, 0.101, 0.098], [0.219, 0.241, 0.222, 0.
118, 0.102, 0.098], [0.217, 0.242, 0.228, 0.115, 0.101, 0.096]]'}  282
s
03:56:10 SEDFx-OOD        lora         0.7946 +/- 0.0014  {'epochs':
'[4, 4, 4]', 'gate': '[0.111, -0.115, 0.113]'}  1371s
03:56:10 SEDFx-OOD done in 105.7 min, saved /kaggle/working/results/SE
DFx-OOD_results.csv
  dataset     method   mean    std                 per_seed
SEDFx-OOD         rf 0.7640 0.0002  [0.764, 0.7642, 0.7638]
SEDFx-OOD     logreg 0.7840 0.0000                  [0.784]
SEDFx-OOD       head 0.7928 0.0004 [0.7928, 0.7922, 0.7933]
SEDFx-OOD      mixer 0.7979 0.0006 [0.7987, 0.7975, 0.7974]
SEDFx-OOD       pool 0.7909 0.0006 [0.7901, 0.7912, 0.7914]
SEDFx-OOD pool_mixer 0.7962 0.0002  [0.7962, 0.7964, 0.796]
SEDFx-OOD       lora 0.7946 0.0014  [0.7932, 0.7965, 0.794]
method       head  logreg    lora   mixer    pool  pool_mixer     rf
dataset
SEDFx-OOD  0.7928   0.784  0.7946  0.7979  0.7909      0.7962  0.764
```